Daniela Rocío Pineda Pineda

Arquitecturas de Software Robustas

Jueves, 12 de noviembre del 2026

EXP-008 - HU5.1.2: Cuota por socio en el API Gateway

Con una capacidad calibrada de C = 60 cotizaciones por segundo, el socio D subió de 9 a 90 cotizaciones por segundo (10x) mientras A, B y C seguían cotizando a 9 cada uno.

Sin cuota, el pico de D dejó sin servicio a todos: en las 3 corridas, A, B y C tuvieron entre 85,7 % y 88,8 % de error. Con la cuota de 12 por segundo por socio, A, B y C tuvieron un p95 entre 43 ms y 86 ms y casi ningún error, y a D se le aceptaron 12 peticiones por segundo mientras el resto recibía 429.

La hipótesis queda parcialmente validada. El p95 cumple con mucho margen, pero el p99 pasó de 500 ms en 4 de 9 mediciones, por pausas de Cotización que se ven también antes de que D empiece a subir.

1. Experimento diseñado

El experimento quedó registrado en Hélix en la entrega intermedia. Lo resumo aquí para poder comparar los resultados con lo que se planteó.

Elemento del ASR	**HU5.1.2 (escalabilidad)**
Fuente	Un socio de distribución (socio D)
Estímulo	Sube su tráfico de 1x a 10x en un minuto
Artefacto	API Gateway (Fachada), Cotización y Perfilamiento, en el flujo de cotización embebida
Ambiente	Operación normal, con otros tres socios (A, B y C) cotizando a ritmo constante
Respuesta	El pico de D no degrada a los demás socios
Medida	A, B y C mantienen p95 < 250 ms y p99 < 500 ms

La hipótesis registrada fue esta:

> *Si el API Gateway aplica una cuota por socio con token bucket, en la que cada socio se identifica por su credencial y el exceso recibe un 429, entonces, cuando un socio sube su tráfico de 1x a 10x en un minuto, las cotizaciones de los demás socios mantienen un p95 menor a 250 ms y un p99 menor a 500 ms. Esto ocurre porque el exceso del socio en pico se descarta en el gateway antes de consumir capacidad de Cotización y Perfilamiento. Sin la cuota, con la misma capacidad, el p95 de los demás socios supera los 250 ms durante el pico.*

La táctica es la limitación de tasa por socio con un token bucket guardado en Redis. Como cada socio tiene su propio contador, la táctica también aísla la carga de uno frente a los otros. Se aplica dentro del patrón API Gateway, que es la única puerta de entrada de los socios, y el exceso se rechaza con 429 y el encabezado Retry-After.

Variable	Definición
Independiente	Cuota por socio activada o desactivada con RATE_LIMIT_ENABLED
Dependientes	p95, p99 y % de error de A, B y C (la medida del ASR) Peticiones aceptadas y rechazadas con 429 del socio D
Diagnóstico	CPU y memoria de cada tarea, CPU y conexiones de cada base de datos, CPU de Redis
Controladas	Una tarea por servicio, sin autoescalado, con los mismos recursos (0,5 vCPU y 1 GB) Los mismos 10.000 perfiles de riesgo y el mismo producto El mismo perfil de carga y el mismo generador Los flags de los demás experimentos apagados

Primero se calibra sin cuota con carga escalonada. La capacidad C es la última carga con p95 < 250 ms y error < 1 %. Cada corrida dura 14 minutos y sigue este perfil:

- A, B y C envían 0,15C cada uno durante toda la corrida.
- D arranca en 0,15C, a los 3 minutos sube en un minuto hasta 1,5C (10x), se queda en el pico 6 minutos, baja en un minuto y termina con 3 minutos de recuperación.
- La cuota es de 0,2C por socio, con una ráfaga de 2 segundos.
- Se hacen 3 corridas sin cuota y 3 con cuota, alternadas, y entre una y otra se reinicia todo.
- Se mide entre los minutos 3 y 10, que cubren la rampa y el pico de D. Si A, B o C reciben un error, esa petición cuenta como fuera de la medida.

La hipótesis se confirma si se cumplen tres condiciones:

1. Con cuota, A, B y C mantienen p95 < 250 ms y p99 < 500 ms en las 3 corridas.
2. Sin cuota, al menos un socio normal supera el p95 de 250 ms.
3. Con cuota, lo que se le acepta a D queda dentro de su cuota y el resto recibe 429.

2. Montaje y ejecución

2.1 Despliegue en AWS

Toda la infraestructura sale de la plantilla de CloudFormation experimento/infra/exp008.yaml. Las imágenes se construyeron desde la rama Solventa_Microservicios (commit 205ed11) con los cambios de este experimento.

Componente	Configuración usada
API Gateway	NestJS en ECS Fargate, 1 tarea de 0,5 vCPU y 1 GB, detrás de un ALB público. Aquí vive la cuota por socio.
Cotización	FastAPI en ECS Fargate, 1 tarea de 0,5 vCPU y 1 GB, detrás de un ALB interno, con su propia RDS PostgreSQL 16.15 (db.t3.micro).
Perfilamiento	FastAPI en ECS Fargate, 1 tarea de 0,5 vCPU y 1 GB, detrás del ALB interno, con su propia RDS PostgreSQL 16.15 (db.t3.micro) y 10.000 perfiles sembrados.
Redis de cuotas	ElastiCache Redis 7.1, cache.t3.micro

Componente	Configuración usada
Generador	EC2 m5.large con Apache JMeter 5.6.3 y un Open Model Thread Group por socio. Entra por el ALB público, igual que un socio real.
Flujo	Socio → ALB público → API Gateway → ALB interno → Cotización → ALB interno → Perfilamiento. Cotización pide el score del cliente, calcula la prima y guarda la cotización.

2.2 Implementación de la táctica

Archivo	Qué hace
services/api-gateway/src/cuota-socio/cuota-socio.guard.ts	Es un guard global. Si RATE_LIMIT_ENABLED no vale true, deja pasar todo. Solo limita las rutas /v1/, así que /health queda libre. Identifica al socio por x-api-key y solo guarda un hash de la clave. Cuando el socio se queda sin fichas, responde 429 con Retry-After. Si Redis falla o tarda más de 50 ms, deja pasar la petición (fail-open). Cada 10 s escribe en el log cuántas peticiones aceptó y rechazó por socio.
services/api-gateway/src/cuota-socio/token-bucket.ts	El token bucket es un script Lua que Redis ejecuta de forma atómica. Rellena las fichas según el tiempo que pasó (con la hora de Redis), gasta una y devuelve si la petición pasa y cuánto hay que esperar.
services/api-gateway/src/cuota-socio/cuota-socio.module.ts y app.module.ts	Registran el guard como APP_GUARD. En app.module.ts el cambio es una sola línea.
Task definition del gateway	Define RATE_LIMIT_ENABLED, RATE_LIMIT_RPS (0,2C), RATE_LIMIT_BURST_S (2) y el REDIS_URL del ElastiCache. Hay una revisión con el flag en false (la 10) y otra en true (la 11).

Con el flag apagado, el gateway se comporta igual que la base compartida del equipo, y la cadena de prueba sigue devolviendo la prima de 172.5 para el cliente demo.

2.3 Cronología

Hora (Colombia, 11 de noviembre)	Paso	Resultado
20:51	Siembra de 10.000 perfiles por POST /perfiles	Todos con 201
20:55 a 20:57	Prueba de humo (1 min, 5 por segundo por socio)	1.200 peticiones
21:01 a 21:14	Calibración 1 (escalones de 20)	Se detuvo en el escalón de 140: desde 80 el sistema ya había colapsado
21:19 a 21:28	Corrida piloto sin cuota con C = 20	Se detuvo en el minuto 8 (ver 2.4)
21:32 a 21:41	Calibración 2 (escalones de 10, de 30 a 60)	21.600 peticiones

Hora (Colombia, 11 de noviembre)	Paso	Resultado
21:46 a 22:01	Corrida 1 sin cuota	64.260 peticiones
22:15 a 22:30	Corrida 2 con cuota	64.260 peticiones
22:34 a 22:50	Corrida 3 sin cuota	64.260 peticiones
22:54 a 23:09	Corrida 4 con cuota	64.260 peticiones
23:14 a 23:29	Corrida 5 sin cuota	64.260 peticiones
23:33 a 23:48	Corrida 6 con cuota	64.260 peticiones

2.4 Cambios frente al diseño

- La calibración se hizo dos veces. La primera, con escalones de 20 como estaba diseñada, dio C = 20. Con ese valor hice una corrida piloto y el pico de D no alcanzó a saturar el sistema, así que no había forma de probar la condición 2. Por eso repetí la calibración con escalones de 10 entre 30 y 60, para ubicar mejor el punto de quiebre. En el informe están las dos.
- La primera calibración la detuve en el escalón de 140, porque desde 80 el sistema ya había colapsado. La corrida piloto la detuve en el minuto 8, cuando ya se veía que no saturaba.
- La corrida 2 se tuvo que relanzar. La primera vez, el gateway con cuota no arrancó porque la plantilla le pasaba vacía la dirección de Redis. Corregí la plantilla. La corrida 1 no se vio afectada, porque sin cuota el gateway no usa Redis.
- Las corridas largas las lanzó un script en la misma EC2 de JMeter (runner_ec2.sh) y no un equipo local. Eso no cambia la carga, solo evita depender de que mi equipo siga conectado.

3. Resultados

3.1 Calibración

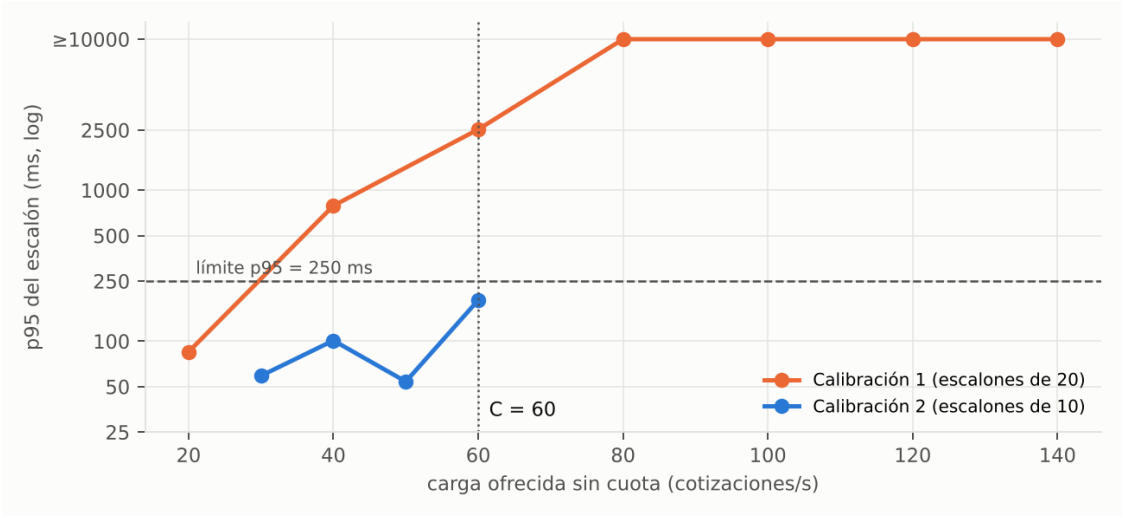
La primera calibración, con escalones de 20 cotizaciones por segundo, dio C = 20, porque en el escalón de 40 el p95 subió a 789 ms. Pero en la corrida piloto con C = 20 el sistema atendió 39 cotizaciones por segundo y el p95 de A, B y C no pasó de 85 ms. Con ese C el pico de D no saturaba nada.

En la segunda calibración, con escalones de 10 entre 30 y 60, todos los escalones cumplieron, con p95 entre 54 ms y 187 ms. Tomé C = 60 porque es el último escalón medido que cumple. Además, la primera calibración ya había mostrado que en 80 el sistema colapsa (84,5 % de error).

Escalón (cotizaciones/s)	Peticiones	Exitosas por s	p95 (ms)	p99 (ms)	Error %	Cumple
Calibración 1: 20	2.400	20	85	345	0,1	Sí
Calibración 1: 40	4.800	40	789	1.830	0,1	No
Calibración 1: 60	7.200	59,6	2.527	2.993	0,8	No
Calibración 1: 80	9.600	12,4	fuera (fallo)	fuera (fallo)	84,5	No
Calibración 1: 100	12.000	9,3	fuera (fallo)	fuera (fallo)	90,7	No

Escalón (cotizaciones/s)	Peticiones	Exitosas por s	p95 (ms)	p99 (ms)	Error %	Cumple
Calibración 1: 120	14.400	0	fuera (fallo)	fuera (fallo)	100	No
Calibración 1: 140	4.158	0	fuera (fallo)	fuera (fallo)	100	No
Calibración 2: 30	3.600	30,1	59	288	0	Sí
Calibración 2: 40	4.800	40	101	550	0	Sí
Calibración 2: 50	6.000	50	54	379	0	Sí
Calibración 2: 60	7.200	60	187	691	0	Sí

La gráfica compara el p95 de cada escalón en las dos calibraciones, en escala logarítmica. Los escalones donde casi todo falló aparecen arriba, en 10 s, que es el tiempo máximo que espera JMeter.



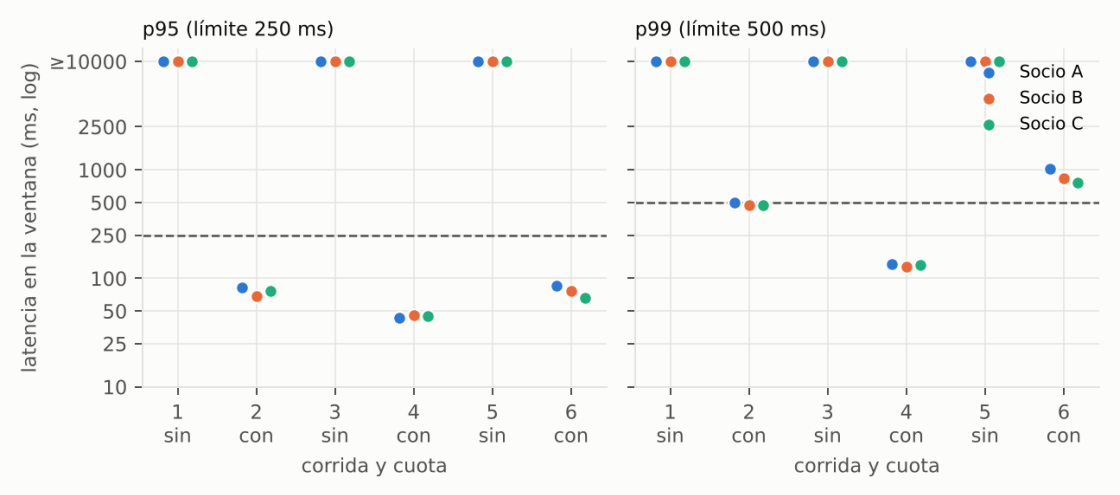
3.2 Socios normales A, B y C

Con C = 60, A, B y C enviaron 9 cotizaciones por segundo cada uno y la cuota quedó en 12 por segundo por socio. Los datos son de la ventana de medición, entre los minutos 3 y 10 de cada corrida.

Corrida	Cuota	Socio	Peticiones	p50 (ms)	p95 (ms)	p99 (ms)	Error %	Cumple el ASR
1	sin	A	3.789	fuera (fallo)	fuera (fallo)	fuera (fallo)	87,9	No
1	sin	B	3.752	fuera (fallo)	fuera (fallo)	fuera (fallo)	88,6	No
1	sin	C	3.842	fuera (fallo)	fuera (fallo)	fuera (fallo)	88,8	No
2	con	A	3.724	22	83	504	0	No
2	con	B	3.893	22	69	470	0	Sí

Corrida	Cuota	Socio	Peticiones	p50 (ms)	p95 (ms)	p99 (ms)	Error %	Cumple el ASR
2	con	C	3.846	22	77	477	0	Sí
3	sin	A	3.844	fuera (fallo)	fuera (fallo)	fuera (fallo)	87,1	No
3	sin	B	3.791	fuera (fallo)	fuera (fallo)	fuera (fallo)	87,5	No
3	sin	C	3.738	fuera (fallo)	fuera (fallo)	fuera (fallo)	87,4	No
4	con	A	3.785	28	43	136	0	Sí
4	con	B	3.740	28	46	129	0	Sí
4	con	C	3.781	28	45	133	0	Sí
5	sin	A	3.708	fuera (fallo)	fuera (fallo)	fuera (fallo)	85,7	No
5	sin	B	3.830	fuera (fallo)	fuera (fallo)	fuera (fallo)	87,2	No
5	sin	C	3.833	fuera (fallo)	fuera (fallo)	fuera (fallo)	86,3	No
6	con	A	3.819	30	86	1030	0	No
6	con	B	3.749	30	76	840	0	No
6	con	C	3.825	30	66	757	0	No

La gráfica pone el p95 y el p99 de cada socio por corrida, con el límite del ASR como línea punteada. Sin cuota los tres socios quedan arriba porque casi todas sus peticiones fallaron.



3.3 Socio D

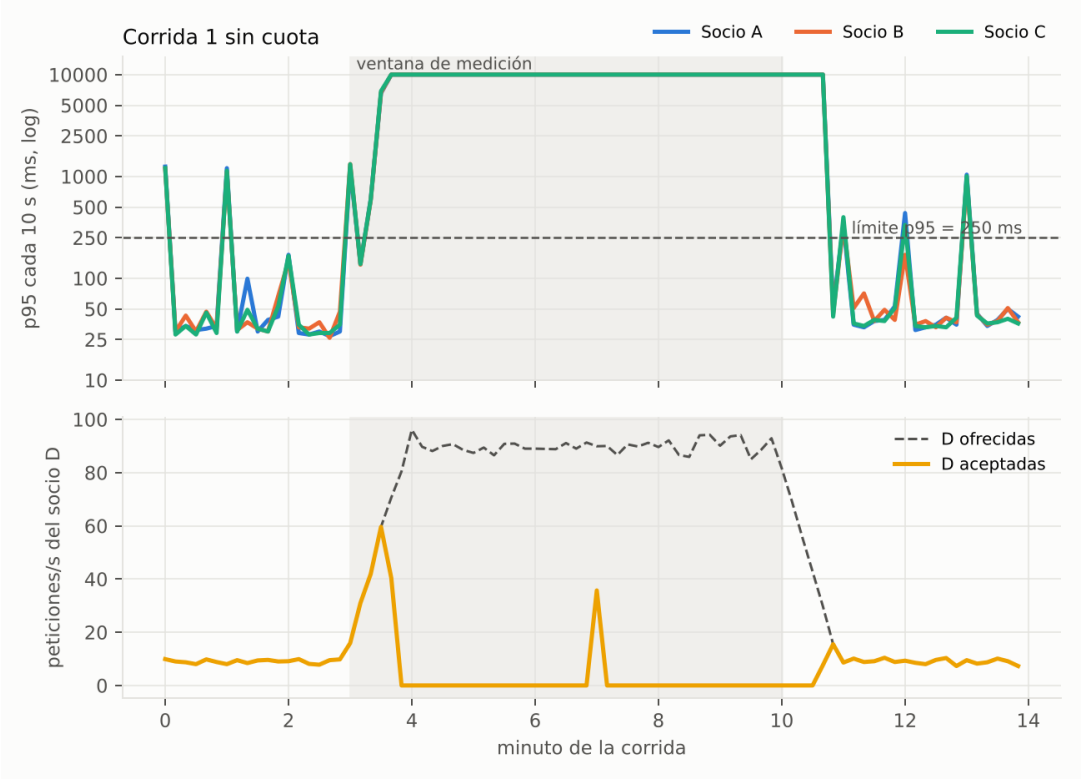
Aquí se mira el pico sostenido de D, entre los minutos 4 y 10. Con cuota, lo máximo que D podía conseguir era la cuota durante 360 s más la ráfaga inicial, es decir 4.344 peticiones.

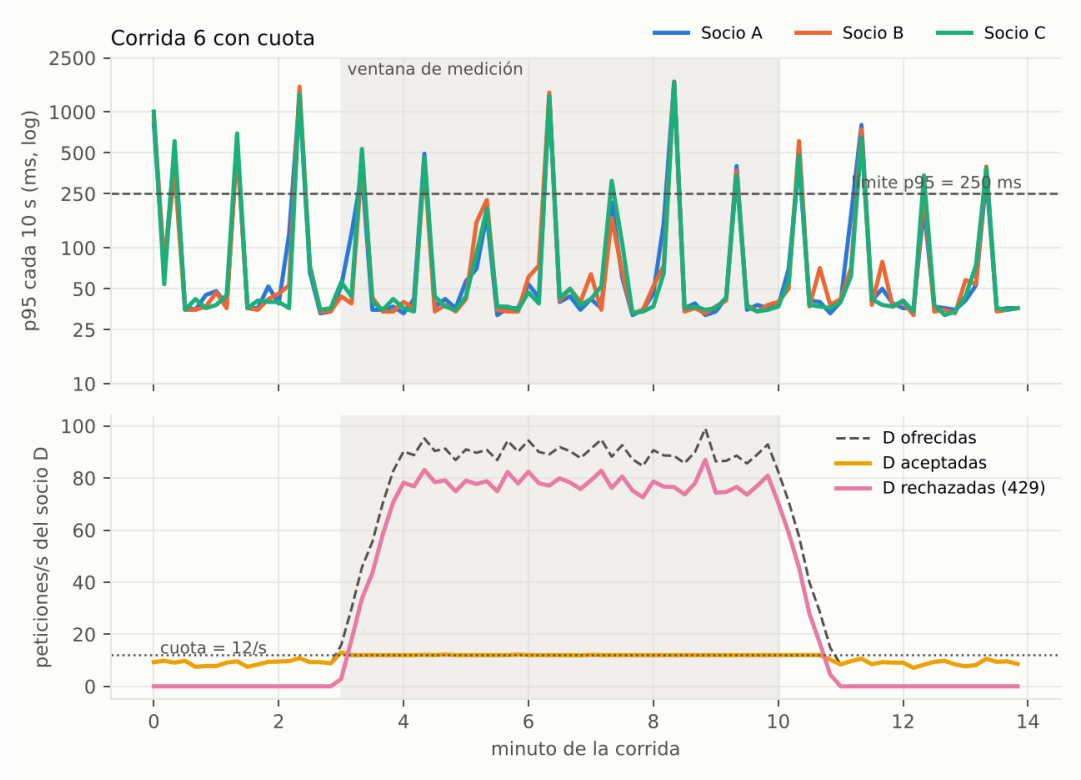
Corrida	Cuota	Ofrecidas por s	Aceptadas por s	Aceptadas	Rechazadas con 429	Otros errores	Máximo aceptado en 1 s
1	sin	90	1	357	0	32.043	117
2	con	90	12	4.333	28.081	2	15
3	sin	90	1,2	426	0	31.972	93
4	con	90	12	4.324	28.070	1	14
5	sin	90	1,5	551	0	31.847	91
6	con	90	12	4.325	28.069	0	14

3.4 Línea de tiempo

Las dos gráficas siguientes son la corrida 1 (sin cuota) y la corrida 6 (con cuota). Arriba está el p95 de A, B y C cada 10 segundos y abajo las peticiones por segundo de D. La franja gris es la ventana de medición. Las demás corridas están en el anexo A.

Sin cuota, la latencia de A, B y C sube junto con la rampa de D y el servicio colapsa mientras dura el pico. Con cuota, las aceptadas de D se quedan planas en 12 por segundo y A, B y C siguen igual que antes del pico. Escogí la corrida 6 porque es la de peor p99: tiene saltos del p95 más o menos cada minuto, y aparecen antes, durante y después del pico de D, siempre con la misma forma. Esos saltos no siguen a D.





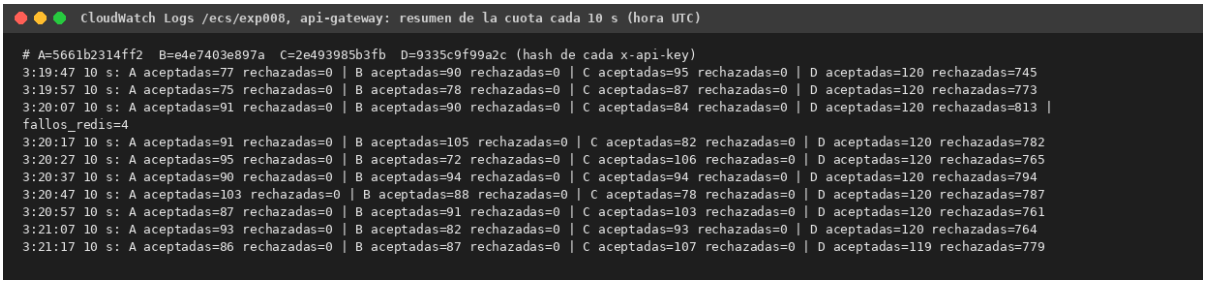
3.5 Diagnóstico en CloudWatch

La tabla muestra el máximo de 1 minuto dentro de la ventana de medición. La CPU de ECS está en porcentaje de los 0,5 vCPU reservados para cada tarea.

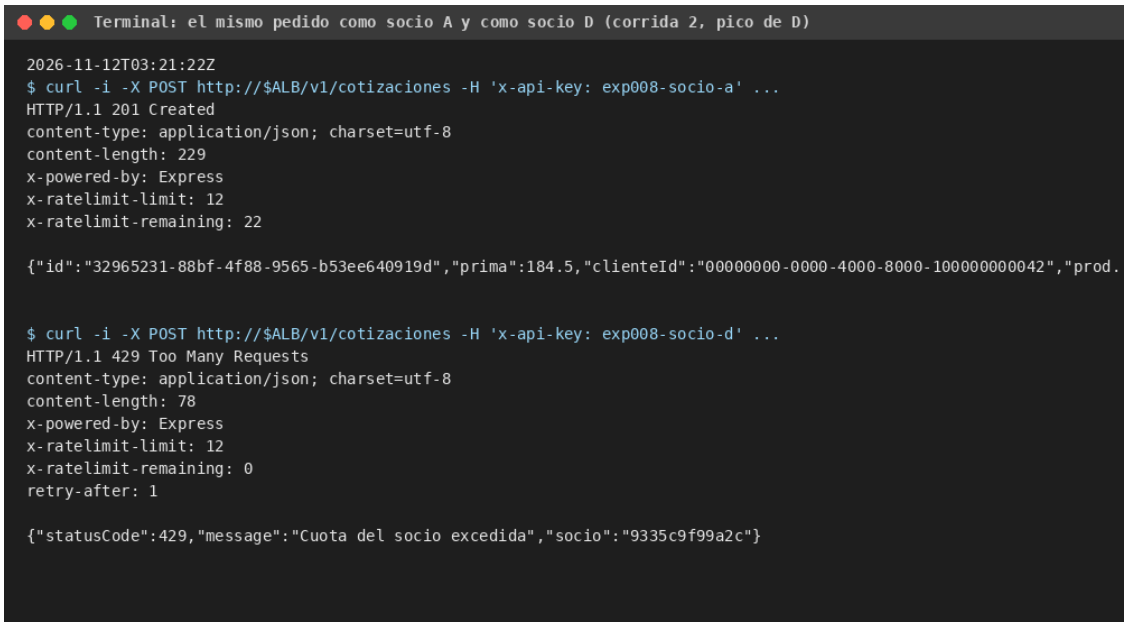
Corrida	Cuota	CPU gateway %	CPU Cotización %	CPU Perfilamiento %	CPU BD Cotización %	Conexiones BD Cotización	CPU BD Perfilamiento %	CPU Redis %	CPU EC2 JMeter %
1	sin	81,5	100	78,7	22,1	20	15,9	2,7	5,5
2	con	36,2	52,1	39,3	8,2	10	8	3,5	4,1
3	sin	58,2	100	81,5	22,7	20	16,3	2,4	5,5
4	con	36,9	53,9	32,3	8,3	10	7,8	4	3
5	sin	88	100	88,4	18,4	20	22,1	2,3	4,4
6	con	35	64,3	34,3	8,6	10	7,4	3,3	3,1

3.6 Evidencia de la táctica funcionando

El gateway deja en CloudWatch un resumen cada 10 segundos con lo que aceptó y rechazó de cada socio. Durante el pico de la corrida 2, D tiene siempre 120 aceptadas, que son exactamente sus 12 por segundo, y entre 760 y 800 rechazadas. A, B y C no tienen ninguna rechazada:



En ese mismo momento hice el mismo pedido como socio A y como socio D. A recibe su cotización con la prima calculada. D recibe 429 con Retry-After: 1 y su petición nunca llega a Cotización:



Sin cuota pasó otra cosa. Durante la corrida 1, Cotización dejó de responder los health checks y ECS tuvo que reemplazar su tarea dos veces en pleno pico:

ECS, eventos de los servicios durante la corrida 1 sin cuota (hora UTC)

```
02:51:41 (service cotizacion) (task 4b76f0f5…) (port 8000) is unhealthy in (target group) due to (reason Request timed out).
02:51:41 (service cotizacion) has stopped 1 running tasks: (task 4b76f0f5…).
02:51:51 (service cotizacion) deregistered 1 targets in (target group)
02:51:51 (service cotizacion) deregistered 1 targets in (target group)
02:51:51 (service cotizacion, taskSet ecs-svc/…) has begun draining connections on 1 tasks.
02:53:30 (service cotizacion) has started 1 tasks: (task 7cbf3593…).
02:53:59 (service cotizacion) registered 1 targets in (target group)
02:54:07 (service cotizacion) has reached a steady state.
02:55:09 (service cotizacion) (task 7cbf3593…) (port 8000) is unhealthy in (target group) due to (reason Request timed out).
02:55:09 (service cotizacion) has stopped 1 running tasks: (task 7cbf3593…).
02:55:18 (service cotizacion) deregistered 1 targets in (target group)
02:55:18 (service cotizacion) deregistered 1 targets in (target group)
02:55:18 (service cotizacion, taskSet ecs-svc/…) has begun draining connections on 1 tasks.
02:57:03 (service cotizacion) has started 1 tasks: (task c79031e4…).
02:57:40 (service cotizacion) registered 1 targets in (target group)
02:57:49 (service cotizacion) has reached a steady state.
```

4. Análisis

4.1 La hipótesis frente a lo observado

Condición de la hipótesis	Lo observado	Resultado
1. Con cuota, A, B y C con p95 < 250 ms y p99 < 500 ms en las 3 corridas	p95 entre 43 ms y 86 ms. p99 entre 129 ms y 1.030 ms. Pasaron de 500 ms el socio A de la corrida 2 (504 ms), el socio A de la corrida 6 (1.030 ms), el socio B de la corrida 6 (840 ms) y el socio C de la corrida 6 (757 ms). Errores de A, B y C: 2 de 34.162 peticiones.	El p95 se cumple, el p99 no en todas
2. Sin cuota, al menos un socio normal con p95 > 250 ms	Los tres socios quedaron fuera de medida en las 3 corridas, con entre 85,7 % y 88,8 % de error.	Se cumple
3. Con cuota, lo aceptado de D dentro de su cuota y el resto con 429	A D se le aceptaron 4.333, 4.324 y 4.325 peticiones en el pico, por debajo del límite de 4.344. De 84.223 no aceptadas, 84.220 recibieron 429 y 3 fueron 502 del balanceador.	Se cumple

Sin cuota, el pico de D llevó la carga total a 117 cotizaciones por segundo, casi el doble de C. Cotización atiende con un solo proceso por tarea y se saturó. Las colas crecieron, las peticiones pasaron de los 10 segundos que espera JMeter y el balanceador empezó a responder 503. Además, el balanceador marcó como no sana la tarea de Cotización porque ni su /health respondía a tiempo, y ECS la reemplazó 2 veces en la corrida 1, 2 veces en la corrida 3 y 2 veces en la corrida 5. Mientras arrancaba la tarea nueva no había destino y todo devolvía 503. En las corridas con cuota no hubo ningún reemplazo. A, B y C no se volvieron un poco más lentos, se quedaron sin servicio. Es la degradación cruzada que el ASR busca evitar, llevada al extremo.

Con cuota, a Cotización le llegó la carga de A, B y C más la cuota de D: 39 cotizaciones por segundo, el 65 % de C. Por eso A, B y C se quedaron con un p50 de entre 22 y 30 ms y un p95 lejos de los 250 ms. El p99 sí se acercó al límite y lo pasó en 4 casos: el socio A de la corrida 2 (504 ms), el socio A de la corrida 6 (1.030 ms), el socio B de la corrida 6 (840 ms) y el socio C de la corrida 6 (757 ms).

Esa cola no la produce D. Antes de que D empiece a subir, entre los minutos 1 y 3, el p99 de A, B y C fue de 530 ms en la corrida 2, 118 ms en la corrida 4 y 1.159 ms en la corrida 6. Con D en pico fue de 482 ms, 131 ms y 909 ms. En las corridas con la cola alta, la cola ya estaba antes del pico y era peor

que durante él. La corrida 4 no tuvo esas pausas y su p99 se quedó en 131 ms aunque D estaba en pleno pico. Son pausas intermitentes de Cotización, las mismas que se vieron en la primera calibración.

En cuanto a D, sus aceptadas quedaron en 12 por segundo en las 3 corridas, que es la cuota configurada, y en ningún segundo pasaron de 15, lo que permite la ráfaga. Hubo 3 respuestas a D que no fueron ni 201 ni 429. Son 502 del balanceador, de unos 5 ms, que aparecen cuando el gateway cierra una conexión que el ALB todavía intenta reutilizar. A, B y C recibieron 2 de esas. No vienen de la cuota y son menos del 0,01 % de lo rechazado, así que doy la condición 3 por cumplida, dejando registrada la excepción.

4.2 Por qué funcionan la táctica y el patrón

La cuota funciona como control de admisión en la entrada. El token bucket decide en el gateway, antes de que empiece cualquier trabajo de negocio, y rechazar una petición cuesta apenas una consulta a Redis. Durante el pico, las respuestas a D tardaron unos 4 ms en la mediana, contra unos 22 ms de una cotización completa. Así se descartaron 78 de las 90 peticiones por segundo de D sin tocar Cotización, Perfilamiento ni sus bases.

Lo que aísla a los socios es que cada uno tiene su propio balde. A, B y C nunca se acercaron a su cuota (mandaban 9 por segundo y tenían 12), así que no recibieron ni un 429. Con un límite global igual a C, los rechazos se habrían repartido entre todos según cuánto mandaba cada uno, y A, B y C habrían perdido cotizaciones sin haber hecho nada distinto.

El patrón API Gateway es lo que permite aplicar esto en un solo punto. Como todos los socios entran por ahí, la regla se pone una vez y no en cada servicio. El costo también queda ahí: el gateway, que recibe todo el exceso de D y lo rechaza, no pasó de 36,9 % de CPU. Sin cuota llegó a 88 %, porque tenía miles de peticiones esperando a Cotización. La CPU máxima de Cotización pasó de 100 % en las corridas sin cuota a entre 52,1 % y 64,3 % con cuota, y Redis no pasó de 4 %. El gateway es fácil de escalar porque no guarda estado, el estado está en Redis.

4.3 Explicaciones alternativas

- Se podría decir que la mejora viene solo de que al backend le llega menos carga. Eso es parte del efecto, pero no lo explica todo. Si fuera solo menos carga, daría igual a quién se le rechaza. Lo que muestra el aislamiento por socio es el reparto: todos los rechazos fueron para D y ninguno para A, B o C.
- Otra posibilidad es que el orden de las corridas o el calentamiento expliquen la diferencia. Las corridas se alternaron (sin, con, sin, con, sin, con) y cada una arrancó con tareas nuevas, Redis vacío y la tabla de cotizaciones vacía. Las tres de cada condición se parecen mucho entre sí y son muy distintas de las de la otra condición.
- El generador podría haber sido el cuello de botella. La CPU de la EC2 de JMeter no pasó de 5,5 %, así que no limitó la carga. Además JMeter usó un modelo abierto: las peticiones llegan aunque las anteriores no hayan terminado, que es lo que pasa con socios reales.
- El p99 alto con cuota podría leerse como degradación causada por D. Pero esa cola ya estaba antes del pico, cuando D mandaba 9 por segundo, y no apareció en la corrida 4. Lo más probable es que Cotización tenga pausas de algunos cientos de milisegundos y que, al correr con un solo proceso, se noten en el 1 % más lento. No llegué a confirmar la causa exacta.
- También podría pensarse que Redis falló y dejó pasar tráfico de más, ya que el gateway deja pasar cuando Redis tarda más de 50 ms. Eso pasó algunas veces (por ejemplo, 4 en una ventana de 10 segundos de la corrida 2), pero las aceptadas de D siguieron en 120 cada 10 segundos. No cambió el resultado.

4.4 Amenazas a la validez

- Validez interna, por la calibración. Las dos calibraciones dieron valores muy distintos (C = 20 y C = 60) sin que cambiara nada del sistema. En la primera hubo pausas que inflaron el p95, y desde el escalón de 80 ECS reemplazó dos veces la tarea de Cotización, lo que también afectó a los escalones siguientes. Si el C real fuera menor que 60, la carga de las corridas con cuota (0,65C) igual quedaría por debajo del colapso que se vio en 80.
- Validez interna, por tener una sola tarea por servicio. Sin autoescalado y con un solo proceso, cualquier pausa se nota mucho en el p99. Es una variable controlada del diseño, pero amplifica la cola.
- Validez de constructo. La latencia se mide en el cliente e incluye el ALB público. Como un fallo cuenta como fuera de la medida, el p95 sin cuota no da un número sino que queda indefinido. Además, la ventana de medición incluye la rampa de D y no solo el pico.
- Validez externa. Los cuatro socios tienen la misma cuota, piden el mismo producto y los datos son sintéticos. En producción habría varias tareas, autoescalado y socios con contratos distintos. El Learner Lab comparte hardware con otros usuarios, lo que puede explicar parte de las pausas.
- Validez de conclusión. Son 3 corridas por condición. La diferencia entre sin cuota y con cuota es enorme y consistente (de casi 90 % de error a casi 0 %), pero el margen del p99 frente a 500 ms es pequeño y con 3 corridas no puedo afirmar que siempre quede por debajo.

5. Conclusión y decisión de arquitectura

La hipótesis queda parcialmente validada. Se cumplió lo que decía sobre el escenario sin cuota: el pico 10x de un socio dejó sin servicio a los demás. También se cumplió lo que decía sobre el socio D: con cuota, su exceso se rechazó en el gateway con 429 y lo aceptado quedó dentro de su cuota. De lo que decía sobre A, B y C, se cumplió el p95, que quedó muy por debajo de 250 ms en las 3 corridas, pero no el p99, que pasó de 500 ms en 4 de 9 mediciones.

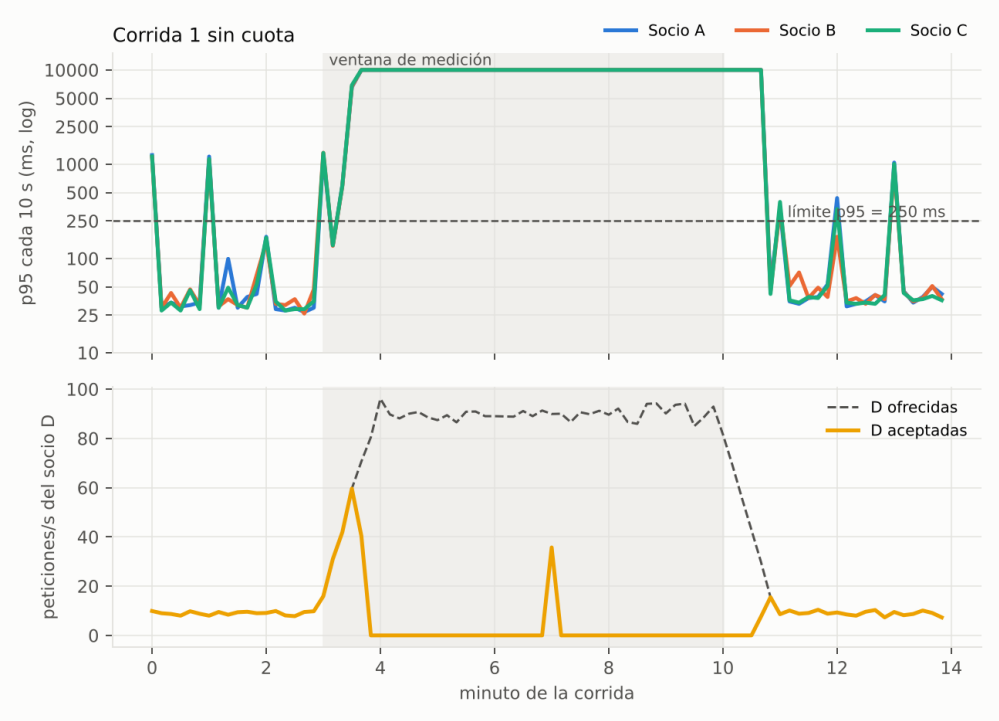
La causa que proponía la hipótesis se sostiene: el exceso del socio en pico se descarta en el gateway antes de gastar capacidad de Cotización y Perfilamiento, y eso es lo que separa un servicio caído de uno que responde con una mediana de entre 22 y 30 ms. Lo que la hipótesis no consideraba es que el p99 tiene una causa propia, que no depende de la carga de los otros socios. Por eso no puedo decir que el ASR se cumple completo con esta infraestructura, aunque la táctica hace lo que se esperaba de ella.

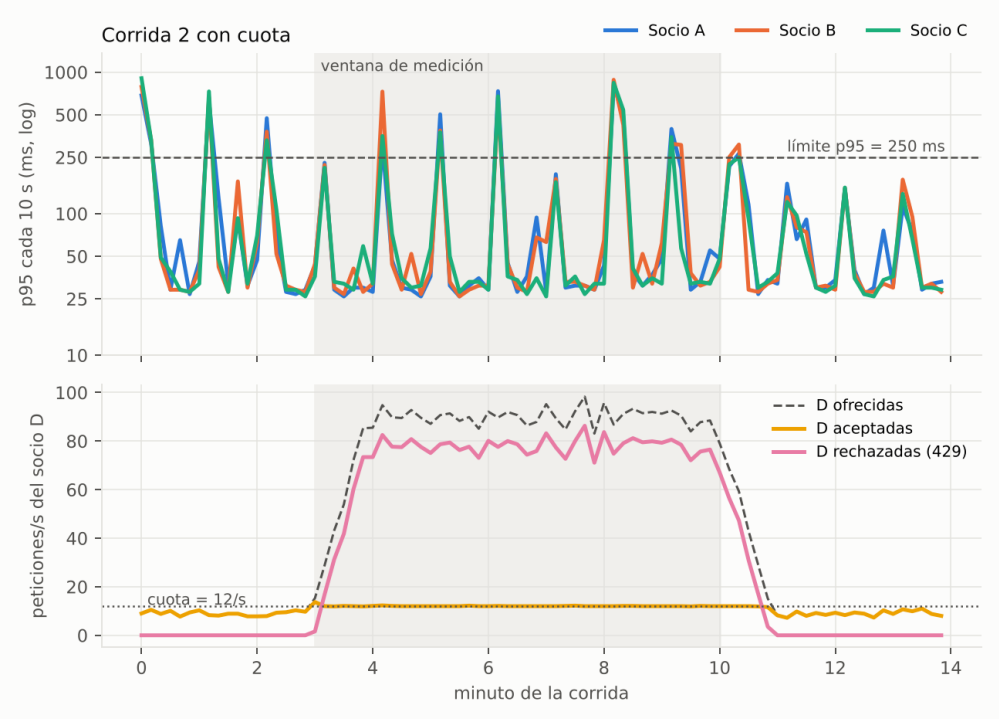
Decisión

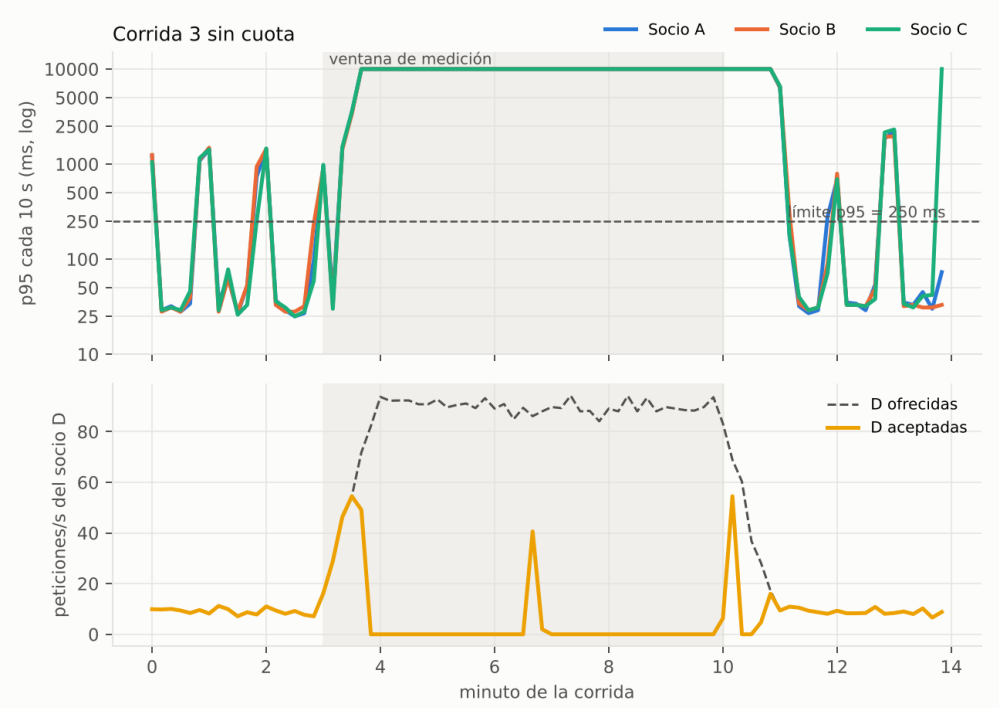
- Adoptar la cuota por socio en el API Gateway, con token bucket en Redis y encendida en producción, con la cuota de cada socio según su contrato y no igual para todos.
- Mantener el fail-open, pero con una alarma sobre el contador de fallos de Redis que ya escribe el gateway. Si Redis falla, es mejor dejar pasar que tumbar las ventas.
- Antes de dar el ASR por cumplido, correr Cotización con al menos 2 tareas o 2 procesos por tarea y buscar el origen de las pausas con un perfilador. También subir el keepAliveTimeout del gateway por encima de los 60 s del ALB, para que desaparezcan los 502.
- Sumar autoescalado y health checks que no dependan del mismo proceso saturado. Sin cuota, ECS reemplazó tareas que estaban vivas pero saturadas, y eso alargó la caída. La cuota evita llegar a ese punto y el autoescalado da margen para los picos legítimos.

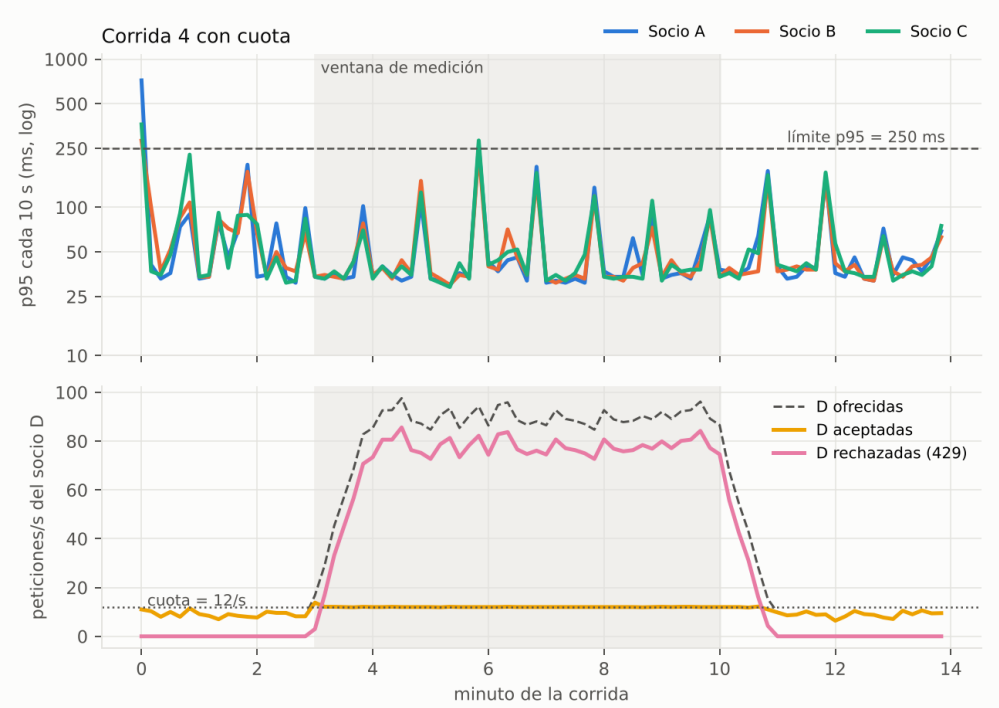
Anexo. Líneas de tiempo de las 6 corridas

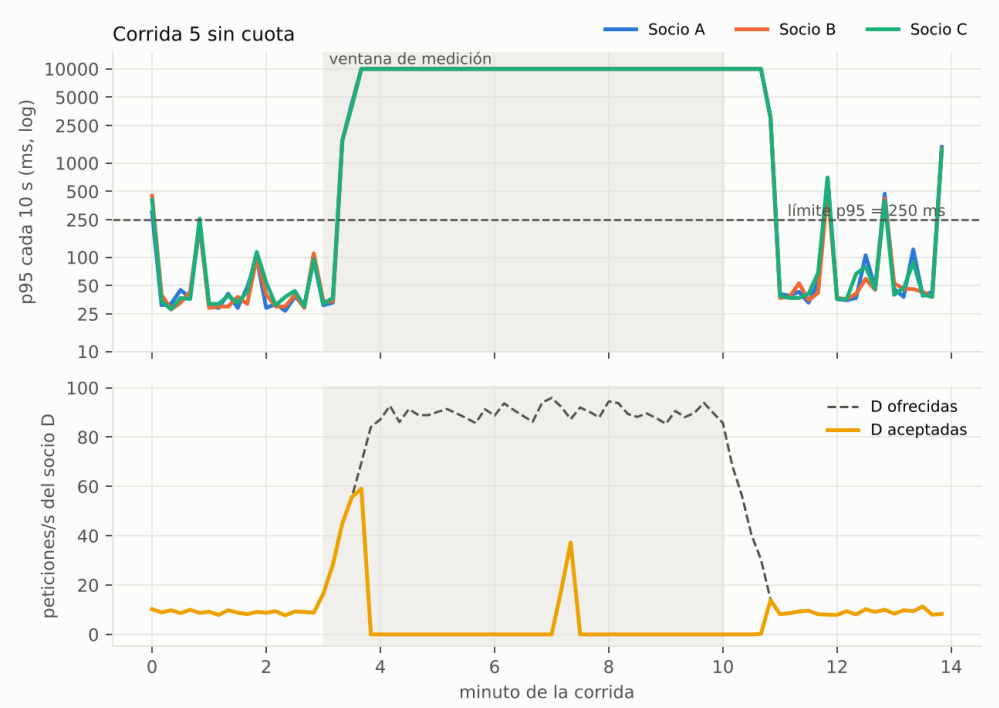
Cada gráfica tiene el mismo formato que las de la sección 3.4. El título de cada una dice la corrida y si llevaba cuota.

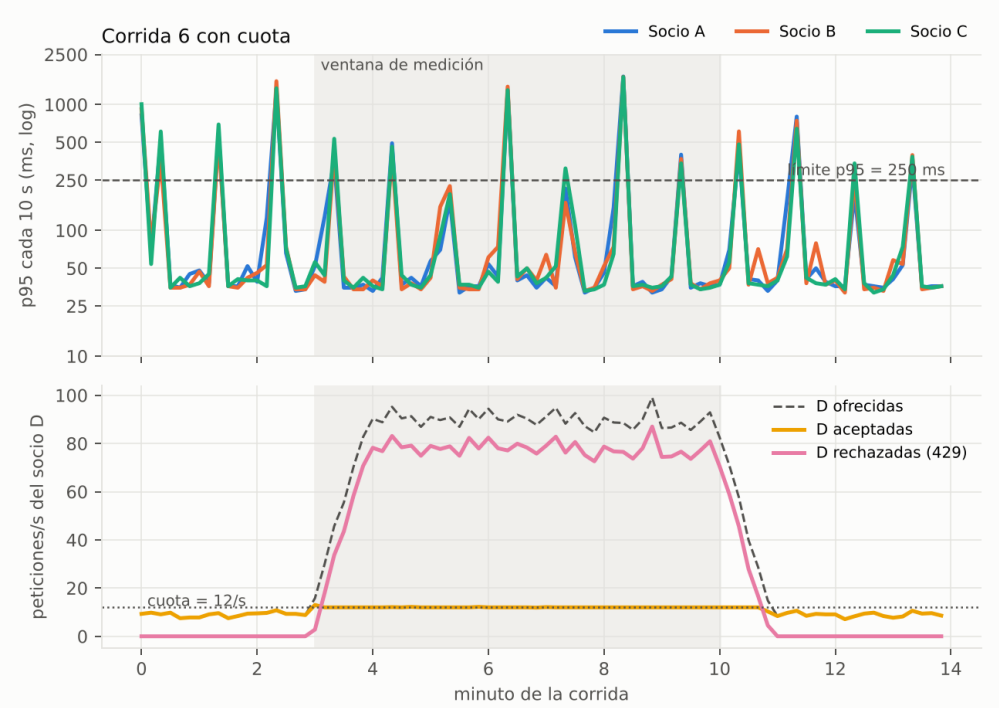





Corrida 3 sin cuota
Socio A
Socio B
Socio C
ventana de medición
límite p95 = 250 ms
p95 cada 10 s (ms, log)
10000
5000
2500
1000
500
250
100
50
25
10
D ofrecidas
D aceptadas
peticiones/s del socio D
80
60
40
20
0
0
2
4
6
8
10
12
14
minuto de la corrida


Corrida 4 con cuota
Socio A
Socio B
Socio C
ventana de medición
límite p95 = 250 ms
p95 cada 10 s (ms, log)
1000
500
250
100
50
25
10
D ofrecidas
D aceptadas
D rechazadas (429)
cuota = 12/s
peticiones/s del socio D
100
80
60
40
20
0
0
2
4
6
8
10
12
14
minuto de la corrida


Corrida 5 sin cuota
Socio A
Socio B
Socio C
ventana de medición
límite p95 = 250 ms
p95 cada 10 s (ms, log)
10000
5000
2500
1000
500
250
100
50
25
10
D ofrecidas
D aceptadas
peticiones/s del socio D
100
80
60
40
20
0
0
2
4
6
8
10
12
14
minuto de la corrida


Corrida 6 con cuota
Socio A
Socio B
Socio C
ventana de medición
límite p95 = 250 ms
p95 cada 10 s (ms, log)
2500
1000
500
250
100
50
25
10
D ofrecidas
D aceptadas
D rechazadas (429)
cuota = 12/s
peticiones/s del socio D
100
80
60
40
20
0
0
2
4
6
8
10
12
14
minuto de la corrida